Chủ đề 04: Kiến trúc Clean Architecture & Dependency Injection

TRƯỜNG ĐẠI HỌC BÁCH KHOA — ĐẠI HỌC ĐÀ NẴNG
KHOA ĐIỆN TỬ VÀ TRÍ TUỆ NHÂN TẠO

Báo cáo Tuần 01 — Môn: Lập trình Đa nền tảng

SVTH: Huỳnh Thiện Lộc – **MSSV:** 106230052

GVHD: TS. Nguyễn Duy Nhật Viễn

Đà Nẵng, 2026

Nội dung báo cáo

1. Phần I: Cơ Sở Lý Thuyết & Nguyên Lý Cốt Lõi
   - Tổng quan Clean Architecture trong Flutter
   - Nguyên lý Đảo ngược Phụ thuộc (DIP)
   - Repository Pattern & UseCase (Interactor)
2. Phần II: Dependency Injection & Khởi Tạo Phụ Thuộc
   - Service Locator với get_it
   - Liên kết với MultiBlocProvider
3. Phần III: Demo Thực Tế & Unit Test Tầng Domain

- Kiến trúc ứng dụng Demo REST API
- Kịch bản Unit Test Tầng Domain với Mocktail
- Tài liệu tham khảo

Phân công công việc (WBS)

TT	HỌ VÀ TÊN	NỘI DUNG ĐẢM NHẬN	TỶ LỆ
1	Huỳnh Thiện Lộc (106230052)	Nghiên cứu Clean Architecture, thiết kế tầng Domain, cấu hình DI (get_it), triển khai tầng Data & Presentation (BLoC), viết Unit Test	100%

Phần I: Cơ Sở Lý Thuyết & Nguyên Lý Cốt Lõi

Tổng quan Clean Architecture trong Flutter

- **Mục tiêu cốt lõi:** Tách biệt các mối bận tâm (**Separation of Concerns**).
- **3 Tầng kiến trúc tiêu chuẩn:**
  1. **Presentation Layer:** Quản lý giao diện (Widgets) và trạng thái (BLoC / Cubit).
  2. **Domain Layer (Trung tâm):** Chứa thực thể nghiệp vụ (Entities), hành động nghiệp vụ (UseCases) và hợp đồng trừu tượng (Repository Contracts). Thuần Dart, độc lập 100%.
  3. **Data Layer:** Triển khai truy xuất dữ liệu thực tế (REST API, SQLite, Cache) và hiện thực Repository từ Domain.

Quy tắc phụ thuộc (The Dependency Rule)

Tầng ngoài phụ thuộc vào tầng trong. Tầng trong **tuyệt đối không biết gì** về tầng ngoài.

Nguyên lý Đảo ngược Phụ thuộc (DIP)

- **Phát biểu (SOLID - ‘D’):**
  1. Module cấp cao không phụ thuộc vào module cấp thấp. Cả hai cùng phụ thuộc vào **trừu tượng (Interface)**.
  2. Giao diện trừu tượng không phụ thuộc chi tiết; chi tiết phụ thuộc vào trừu tượng.

Truyền thống (Tight Coupling):

`Presentation` ⟶ `RemoteDataSource`

Hệ quả: Thay đổi API hoặc client mạng sẽ làm vỡ giao diện.

Áp dụng DIP (Loose Coupling):

`Presentation` ⟶ `PostRepository` (Interface)

`PostRepositoryImpl` ⟶ `PostRepository`

Kết quả: Chiều phụ thuộc mã nguồn đảo ngược so với chiều thực thi.

Repository Pattern & UseCase (Interactor)

UseCase (Interactor):

- Đại diện cho một hành động nghiệp vụ duy nhất (`Single Responsibility`).
- Thực thi qua callable class: `useCase(params)`.
- Trả về kết quả an toàn bằng `Either<Failure, T>` (Functional Error Handling).

Repository Pattern:

- Là cầu nối trừu tượng giữa Domain và Data.
- Che giấu toàn bộ chi tiết kết nối mạng, parsing JSON hay lưu cache.
- Giúp tầng Presentation hoàn toàn độc lập với nguồn gốc dữ liệu.

Phần II: Dependency Injection & Khởi Tạo Phụ Thuộc

Service Locator với get_it

- **Bản chất của DI:** Cung cấp đối tượng phụ thuộc từ bên ngoài vào class thay vì để class tự `new`.
- **Các vòng đời đăng ký trong `get_it`:**
  1. `registerLazySingleton`: Chỉ tạo instance 1 lần khi được gọi lần đầu (dùng cho `HttpClient`, `DataSource`, `Repository`, `UseCase`).
  2. `registerFactory`: Mỗi lần gọi tạo mới 1 instance (dùng cho `BLoC/Cubit` gắn theo widget).

WidgetsFlutterBinding.ensureInitialized()

Bắt buộc gọi trước khi cấu hình DI bất đồng bộ hoặc nạp dịch vụ native trước `runApp()`.

Liên kết với MultiBlocProvider

- **Cơ chế hoạt động:**
  1. `main()` nạp cấu hình `initDependencies()` qua `get_it`.
  2. `MultiBlocProvider` inject instance BLoC vào cây widget của Flutter.
  3. UI Widget lắng nghe qua `BlocBuilder` hoặc `context.read<PostBloc>()`.
- **Lợi ích:** BLoC được tự động giải phóng (`close()`) theo vòng đời của Widget Tree, tránh rò rỉ bộ nhớ (Memory Leak).

Phần III: Demo Thực Tế & Unit Test Tầng Domain

Kiến trúc ứng dụng Demo REST API

- **Bài toán:** Ứng dụng đọc danh sách bài viết từ REST API (`jsonplaceholder.typicode.com/posts`).
- **Luồng thực thi dữ liệu:** `REST API` ⟶ `PostRemoteDataSource` ⟶ `PostModel` ⟶ `PostRepositoryImpl` ⟶ `PostEntity` ⟶ `GetPostsUseCase` ⟶ `PostBloc` ⟶ `PostPage (UI)`.

Bằng chứng Presentation không phụ thuộc Data Source

Tầng Presentation chỉ import `domain/`. Khi đổi DataSource sang Mock hoặc Local Cache, không một dòng code nào trong `presentation/` bị thay đổi.

Kịch bản Unit Test Tầng Domain với Mocktail

- **Kỹ thuật Mocking:** Tạo `MockPostRepository extends Mock implements PostRepository`.
- **Đặc điểm:** Kiểm thử độc lập 100% trong mili-giây, không gọi mạng thật, không dựng widget.

Kịch bản kiểm thử (flutter test)

- **Test 1 (Success):** Mock trả về `Right(List<Post>)` ⟶ UseCase trả về đúng danh sách bài viết, xác minh repository được gọi đúng 1 lần.
- **Test 2 (Failure):** Mock trả về `Left(ServerFailure)` ⟶ UseCase chuyển tiếp failure an toàn, không gây unhandled exception.

Tài liệu tham khảo

1. Robert C. Martin (2017), *Clean Architecture: A Craftsman's Guide to Software Structure and Design*.
2. Flutter Official Documentation: https://docs.flutter.dev
3. Reso Coder, *Flutter Clean Architecture Proposal*.
4. BLoC State Management: https://bloclibrary.dev
5. Mã nguồn Tuần 1: Thư mục `week_01/app` trong repository.